Summary of Source Code

Source code file name	Production code or test code?	# lines of code
GameController.java	Production code	90
Board.java	Production code	208
CellStata.java	Production code	6
DiamondBoard.java	Production Code	34
EnglishBoard.java	Production Code	42
HexagonBoard.java	Production Code	83
Square.java	Production Code	45
BoardView.java	Production Code	175
CellStateTest.java	Test Code	10
DiamondBoardTest.java	Test Code	39
EnglishBoardTest.java	Test Code	287
HexagonBoardTest.java	Test Code	73
SquareTest.java	Test Code	42
	Total	1,134

Production Code vs. User Acceptance Criteria

User Stories

User Story ID	User Story
1	Choose board size and type
2	Start a new game ~~of the chosen board size and type~~
4	Making a move
5	Checking the end of game

User Story ID and Name	AC ID	Class Name(s)	Method Name(s)	Status (complete or not)	Notes

Tests vs. User Stories/Acceptance Criteria

Automated tests directly corresponding to the acceptance criteria of the above user stories

User Story ID and Name	Acceptance Criterion ID	Class Name (s) of the Test Code	Method Name(s) of the Test Code	Status (Pass/Fail/In Progress)
2 -- Start a new game	2.1	EnglishBoardTest	resetBoard	Pass
4 -- Making a move	4.1	EnglishBoardTest	testValidOrthogonal testValidDiagonal testValidDiagonal2 testJumpRemovesPeg testDiagJumpRemovesPeg	Pass
...	4.2	EnglishBoardTest	testInvalidStart testEmptyStart	Pass
...	4.3	EnglishBoardTest	testEmptyJump testEmptyDiagJump	Pass
...	4.4	EnglishBoardTest	testOccupiedLanding	Pass
...	4.5	EnglishBoardTest	testDoubleHop testLongMove	Pass
...	4.6	EnglishBoardTest	testInvalidLanding testOutOfBounds	Pass

...	4.9	EnglishBoardTest	testNeighborMove	Pass
5 -- Checking the end of game	5.1	EnglishBoardTest	testIfNoValidMoves testDetectValidMoves	Pass

Other automated or manual tests not (read: no longer) corresponding to acceptance criteria

No.	**Test Input**	**Expected Result**	**Class Name of the Test Code**	**Method name of the Test Code**
1	CellState.values()	Length of 3	CellStateTest	testExactlyThreeStates
2	Square square = new Square(0, 0, CellState.PEG) square.getState()	CellState.PEG	SquareTest	testInitializePeg
3	Square square = new Square(0, 0, CellState.EMPTY) square.getState()	CellState.EMPTY	SquareTest	testInitializeEmpty
4	Square square = new Square(0, 0, CellState.INVALID) square.getState()	CellState.INVALID	SquareTest	testInitializeInvalid
5	Square square = new Square(0,0, CellState.EMPTY); square.setState(CellState.PEG); square.getState()	CellState.PEG	SquareTest	testSetState
6	Square square = new Square(4, 0, CellState.PEG);	4	SquareTest	testRow

7	Square square = new Square(0,4,CellState.PEG);	4	SquareTest	testColumn
8	EnglishBoard englishBoard = new EnglishBoard(7); englishBoard.getBoardSize()	7	EnglishBoardTest	testBoardSize
9	EnglishBoard englishBoard = new EnglishBoard(7); englishBoard.getGrid()[5][5].getState()) englishBoard.getGrid()[9][5].getState()) englishBoard.getGrid()[5][9].getState()) englishBoard.getGrid()[9][9].getState()) englishBoard.getGrid()[7][7].getState()) englishBoard.getGrid()[5][6].getState()) englishBoard.getGrid()[8][8].getState())	CellState.INVALID CellState.INVALID CellState.INVALID CellState.INVALID CellState.EMPTY CellState.PEG CellState.PEG	EnglishBoardTest	testSizeSevenBoard
10	EnglishBoard englishBoard = new EnglishBoard(15); englishBoard.getGrid()[5][5].getState()) englishBoard.getGrid()[9][5].getState()) englishBoard.getGrid()[5][9].getState()) englishBoard.getGrid()[9][9].getState()) englishBoard.getGrid()[7][7].getState()) englishBoard.getGrid()[5][6].getState()) englishBoard.getGrid()[8][8].getState()) assertEquals(CellState.PEG englishBoard.getGrid()[0][6	CellState.INVALID CellState.INVALID CellState.INVALID CellState.INVALID CellState.EMPTY CellState.PEG CellState.PEG CellState.PEG	EnglishBoardTest	testSizeFifteenBoard

	].getState());			
11	DiamondBoard diamondBoard = new DiamondBoard(7); diamondBoard.getBoardSize();	7	DiamondBoardTest	testBoardSize
12	DiamondBoard diamondBoard = new DiamondBoard(7); diamondBoard.getGrid()[5][5].getState() diamondBoard.getGrid()[9][5].getState() diamondBoard.getGrid()[5][6].getState() diamondBoard.getGrid()[9][6].getState() diamondBoard.getGrid()[7][7].getState() diamondBoard.getGrid()[4][6].getState() diamondBoard.getGrid()[11][7].getState()	CellState.INVALID CellState.INVALID CellState.PEG CellState.PEG CellState.EMPTY CellState.INVALID CellState.INVALID	DiamondBoardTest	testSizeSevenBoard
13	DiamondBoard diamondBoard = new DiamondBoard(7); diamondBoard.getGrid()[0][7].getState()) diamondBoard.getGrid()[7][0].getState()) diamondBoard.getGrid()[14][7].getState()) diamondBoard.getGrid()[7][14].getState()) diamondBoard.getGrid()[7][7].getState()) diamondBoard.getGrid()[0][6].getState()) diamondBoard.getGrid()[0][8].getState())	CellState.PEG CellState.PEG CellState.PEG CellState.PEG CellState.EMPTY CellState.INVALID CellState.INVALID CellState.INVALID CellState.INVALID	DiamondBoardTest	testSizeFifteenBoard

	diamondBoard.getGrid()[6][14].getState()) diamondBoard.getGrid()[8][14].getState())			
14	HexagonBoard hexagonBoard = new HexagonBoard(7); hexagonBoard.getBoardSize()	7	HexagonBoardTest	testBoardSize
15	HexagonBoard hexagonBoard = new HexagonBoard(7) grid[0][0].getState() grid[0][7].getState() grid[0][6].getState() grid[4][7].getState() grid[4][0].getState() grid[4][6].getState() grid[7][6].getState() grid[7][7].getState() grid[7][8].getState() grid[5][7].getState() grid[6][0].getState() grid[7][11].getState() grid[9][10].getState()	CellState.INVALID CellState.INVALID CellState.INVALID CellState.PEG CellState.INVALID CellState.PEG CellState.PEG CellState.EMPTY CellState.PEG CellState.PEG CellState.INVALID CellState.INVALID CellState.INVALID	HexagonBoardTest	testSizeSevenBoard
16	HexagonBoard board = new HexagonBoard(15); Square[][] grid = board.getGrid(); grid[0][0].getState() grid[0][7].getState() grid[0][14].getState()	CellState.INVALID CellState.PEG CellState.INVALID CellState.PEG CellState.INVA	HexagonBoardTest	testSizeFifteenBoard

	grid[3][7].getState() grid[3][0].getState() grid[7][5].getState() grid[7][6].getState() grid[7][7].getState() grid[7][8].getState() grid[7][9].getState() grid[11][7].getState() grid[14][0].getState() grid[14][7].getState() grid[14][14].getState()	LID CellState.PEG CellState.PEG CellState.EMPT Y CellState.PEG CellState.PEG CellState.PEG CellState.INVA LID CellState.PEG CellState.INVA LID		

AI Generated Unit Tests

testSizeSevenBoard() and testSizeFifteenBoard() for the class HexagonBoardTest were generated with the help of ChatGPT.

ChatGPT's versions of the test had a few errors where it was checking squares for different values than they were supposed to have. After fixing that, however, the tests worked fine. In fact, they improved a bit on efficiency by simplifying "hexagonBoard.getGrid()" to a variable "grid."

It is worth noting, however, that I used Claude extensively on this project. I had it ask me questions to direct my thinking; I was still applying critical thinking, but Claude helped me to take problems one at a time and reduce something that seemed complex into a series of simple questions. The longer I worked, the better I got at designing tests by myself.

ChatGPT's output:

```java
@Test
void testSizeSevenBoard() {
    HexagonBoard board = new HexagonBoard(7);
    Square[][] grid = board.getGrid();

    int center = 7 / 2; // GRID_SIZE assumed 7

    // --- Top margin should be invalid
    assertEquals(CellState.INVALID, grid[0][0].getState());
    assertEquals(CellState.INVALID, grid[0][center].getState());
    assertEquals(CellState.INVALID, grid[0][6].getState());

    // --- First playable row
    assertEquals(CellState.PEG, grid[1][center].getState());
    assertEquals(CellState.INVALID, grid[1][0].getState());
    assertEquals(CellState.INVALID, grid[1][6].getState());

    // --- Center row
    assertEquals(CellState.PEG, grid[center][center - 1].getState());
    assertEquals(CellState.EMPTY, grid[center][center].getState());
    assertEquals(CellState.PEG, grid[center][center + 1].getState());

    // --- Bottom playable region
    assertEquals(CellState.PEG, grid[5][center].getState());

    // --- Bottom margin
    assertEquals(CellState.INVALID, grid[6][0].getState());
    assertEquals(CellState.INVALID, grid[6][center].getState());
    assertEquals(CellState.INVALID, grid[6][6].getState());
}
```

```java
@Test
void testSizeFifteenBoard() {
    HexagonBoard board = new HexagonBoard(15);
    Square[][] grid = board.getGrid();

    int center = 15 / 2; // 7

    // --- Top margin invalid
    assertEquals(CellState.INVALID, grid[0][0].getState());
    assertEquals(CellState.INVALID, grid[0][center].getState());
    assertEquals(CellState.INVALID, grid[0][14].getState());

    // --- Early playable row
    assertEquals(CellState.PEG, grid[3][center].getState());
    assertEquals(CellState.INVALID, grid[3][0].getState());

    // --- Center row checks
    assertEquals(CellState.PEG, grid[center][center - 2].getState());
    assertEquals(CellState.PEG, grid[center][center - 1].getState());
    assertEquals(CellState.EMPTY, grid[center][center].getState());
    assertEquals(CellState.PEG, grid[center][center + 1].getState());
    assertEquals(CellState.PEG, grid[center][center + 2].getState());

    // --- Lower playable region
    assertEquals(CellState.PEG, grid[11][center].getState());

    // --- Bottom margin
    assertEquals(CellState.INVALID, grid[14][0].getState());
    assertEquals(CellState.INVALID, grid[14][center].getState());
    assertEquals(CellState.INVALID, grid[14][14].getState());
}
```

The modified version of it in my code:

```java
@Test
void testSizeSevenBoard() {
    HexagonBoard hexagonBoard = new HexagonBoard(7);
    Square[][] grid = hexagonBoard.getGrid();
    // --- Top margin should be invalid
    assertEquals(CellState.INVALID, grid[0][0].getState());
    assertEquals(CellState.INVALID, grid[0][7].getState());
    assertEquals(CellState.INVALID, grid[0][6].getState());
    // --- First playable row
```

```
    assertEquals(CellState.PEG, grid[4][7].getState());
    assertEquals(CellState.INVALID, grid[4][0].getState());
    assertEquals(CellState.PEG, grid[4][6].getState());
    // --- Center row
    assertEquals(CellState.PEG, grid[7][6].getState());
    assertEquals(CellState.EMPTY, grid[7][7].getState());
    assertEquals(CellState.PEG, grid[7][8].getState());
    // --- Bottom playable region
    assertEquals(CellState.PEG, grid[5][7].getState());
    // --- Bottom margin
    assertEquals(CellState.INVALID, grid[6][0].getState());
    assertEquals(CellState.INVALID, grid[7][11].getState());
    assertEquals(CellState.INVALID, grid[9][10].getState());
}

@Test
void testSizeFifteenBoard() {
    HexagonBoard board = new HexagonBoard(15);
    Square[][] grid = board.getGrid();
    // --- Top margin invalid
    assertEquals(CellState.INVALID, grid[0][0].getState());
    assertEquals(CellState.PEG, grid[0][7].getState());
    assertEquals(CellState.INVALID, grid[0][14].getState());
    // --- Early playable row
    assertEquals(CellState.PEG, grid[3][7].getState());
    assertEquals(CellState.INVALID, grid[3][0].getState());
    // --- Center row checks
    assertEquals(CellState.PEG, grid[7][5].getState());
    assertEquals(CellState.PEG, grid[7][6].getState());
    assertEquals(CellState.EMPTY, grid[7][7].getState());
    assertEquals(CellState.PEG, grid[7][8].getState());
    assertEquals(CellState.PEG, grid[7][9].getState());
    // --- Lower playable region
    assertEquals(CellState.PEG, grid[11][7].getState());
    // --- Bottom margin
    assertEquals(CellState.INVALID, grid[14][0].getState());
    assertEquals(CellState.PEG, grid[14][7].getState());
    assertEquals(CellState.INVALID, grid[14][14].getState());
}
```

ChatGPT conversation link for generating tests:
https://chatgpt.com/share/69af267e-3938-8004-8fed-a7059222061e

Claude conversation link for working through the assignment:
https://claude.ai/share/563c5b89-3884-45d0-ad46-ce441b2d0392